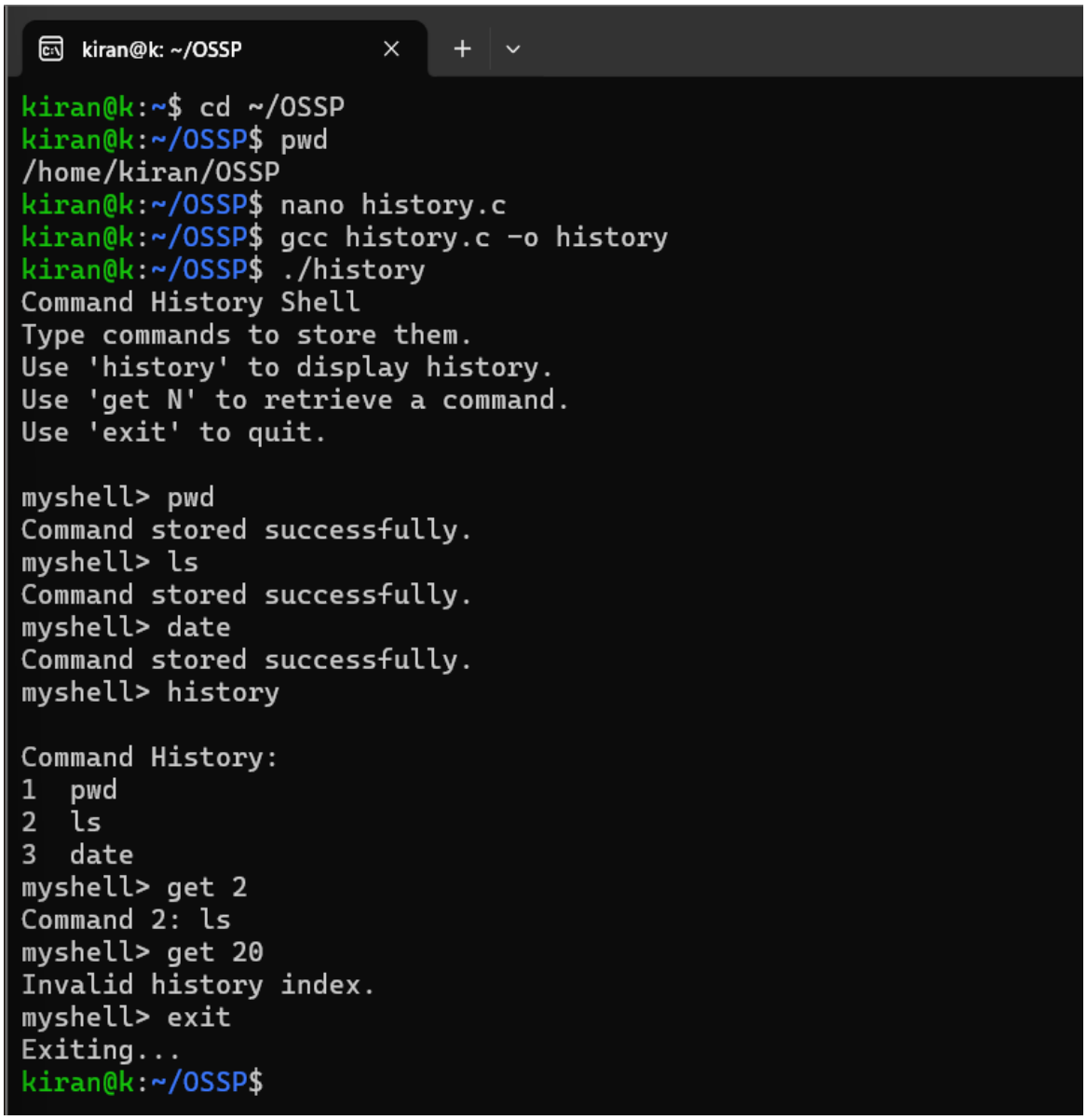
Skill Week-6

Session-11

Task 1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency



```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define HISTORY_SIZE 5
#define MAX_COMMAND 256

char history[HISTORY_SIZE][MAX_COMMAND];

int count = 0;
int start = 0;

/* Add command to history */
void add_history(const char *command) {

    int position;

    if (count < HISTORY_SIZE) {

        position = (start + count) % HISTORY_SIZE;

        strcpy(history[position], command);

        count++;
    }
    else {

        /* Buffer is full */
        position = start;

        strcpy(history[position], command);

        start = (start + 1) % HISTORY_SIZE;
    }
}

/* Display history */
void display_history() {

    if (count == 0) {
        printf("History is empty.\n");
        return;
    }

    printf("\nCommand History:\n");

    for (int i = 0; i < count; i++) {

        int position = (start + i) % HISTORY_SIZE;

        printf("%d  %s\n", i + 1, history[position]);
```

```
  GNU nano 8.7.1
    }
}

/* Retrieve command */
void retrieve_command(int number) {

    if (number < 1 || number > count) {
        printf("Invalid history number.\n");
        return;
    }

    int position = (start + number - 1) % HISTORY_SIZE;

    printf("Command %d: %s\n",
           number, history[position]);
}

/* Validate history */
void validate_history() {

    if (count < 0 || count > HISTORY_SIZE) {
        printf("History validation failed.\n");
        return;
    }

    printf("History validation successful.\n");
}

int main() {

    char command[MAX_COMMAND];

    printf("Command History Manager\n");
    printf("History capacity: %d commands\n", HISTORY_SIZE);
    printf("Type 'history' to display history.\n");
    printf("Type 'get N' to retrieve a command.\n");
    printf("Type 'check' to validate history.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {

        printf("myshell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0) {
            break;
        }
```

```
        if (strcmp(command, "exit") == 0) {
            break;
        }

        if (strcmp(command, "history") == 0) {
            display_history();
            continue;
        }

        if (strcmp(command, "check") == 0) {
            validate_history();
            continue;
        }

        if (strncmp(command, "get ", 4) == 0) {

            int number = atoi(command + 4);

            retrieve_command(number);
            continue;
        }

        if (strlen(command) > 0) {
            add_history(command);
        }
    }

    printf("History manager exited.\n");

    return 0;
}
```

Task 2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
kiran@k:~/OSSP$ nano pipeline.c
kiran@k:~/OSSP$ gcc pipeline.c -o pipeline
kiran@k:~/OSSP$ ./pipeline
Pipeline Command Shell
Enter commands to build a pipeline.
Use 'show' to display pipeline.
Use 'clear' to reset pipeline.
Use 'exit' to quit.

myshell> ls
Command stored successfully.
myshell> grep ".c"
Command stored successfully.
myshell> wc -l
Command stored successfully.
myshell> show

Pipeline Layout:
1: ls
  |
2: grep ".c"
  |
3: wc -l

myshell> clear
Pipeline cleared.
myshell> show
Pipeline is empty.
myshell> exit
Exiting...
kiran@k:~/OSSP$
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COMMANDS 10
#define MAX_LENGTH 256

struct Pipeline {
    char commands[MAX_COMMANDS][MAX_LENGTH];
    int count;
};

/* Add command to pipeline */
void add_command(struct Pipeline *pipeline,
                 const char *command) {

    if (pipeline->count >= MAX_COMMANDS) {
        printf("Pipeline capacity exceeded.\n");
        return;
    }

    strcpy(pipeline->commands[pipeline->count],
           command);

    pipeline->count++;
}

/* Display pipeline */
void display_pipeline(struct Pipeline *pipeline) {

    printf("\nPipeline:\n");

    for (int i = 0; i < pipeline->count; i++) {

        printf("Command %d: %s\n",
               i + 1,
               pipeline->commands[i]);

        if (i < pipeline->count - 1)
            printf("         |\n");
    }
}

/* Validate pipeline */
int validate_pipeline(struct Pipeline *pipeline) {

    if (pipeline->count == 0) {
        printf("Pipeline is empty.\n");
        return 0;
    }

    if (pipeline->count > MAX_COMMANDS) {
```

```
  GNU nano 8.7.1
    if (pipeline->count > MAX_COMMANDS) {
        printf("Too many commands.\n");
        return 0;
    }

    for (int i = 0; i < pipeline->count; i++) {

        if (strlen(pipeline->commands[i]) == 0) {
            printf("Invalid empty command.\n");
            return 0;
        }
    }

    printf("Pipeline layout is valid.\n");

    return 1;
}

int main() {

    struct Pipeline pipeline;

    pipeline.count = 0;

    char input[MAX_LENGTH];

    printf("Pipeline Structure Manager\n");
    printf("Enter commands one by one.\n");
    printf("Type 'done' when finished.\n\n");

    while (pipeline.count < MAX_COMMANDS) {

        printf("Command %d: ",
               pipeline.count + 1);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "done") == 0)
            break;

        if (strlen(input) == 0) {
            printf("Empty command is not allowed.\n");
            continue;
        }

        add_command(&pipeline, input);
    }

    display_pipeline(&pipeline);

    validate_pipeline(&pipeline);

    return 0;
}
```

Session-12

Task 1: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```
kiran@k:~/OSSP$ nano pipe_exec.c
kiran@k:~/OSSP$ gcc pipe_exec.c -o pipe_exec
kiran@k:~/OSSP$ ./pipe_exec
49
kiran@k:~/OSSP$ ./pipe_exec
49
kiran@k:~/OSSP$ nano pipeline.c
kiran@k:~/OSSP$ gcc pipeline.c -o pipeline
kiran@k:~/OSSP$ ./pipeline
Pipeline Structure Tester
Enter commands separated by |
Example: ls | wc
Type exit to quit.

myshell> ls | grep c | wc

Pipeline Layout:
Command 1: ls
       |
Command 2: grep c
       |
Command 3: wc


Total commands: 3
Pipeline structure is valid.
myshell> exit
Exiting...
kiran@k:~/OSSP$
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    int pipefd[2];

    pid_t child1, child2;

    /* Create pipe */
    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    /* First child */
    child1 = fork();

    if (child1 < 0) {
        perror("fork");
        return 1;
    }

    if (child1 == 0) {

        /* Redirect stdout to pipe */
        dup2(pipefd[1], STDOUT_FILENO);

        /* Close unused descriptors */
        close(pipefd[0]);
        close(pipefd[1]);

        /* Execute ls */
        execlp("ls", "ls", NULL);

        perror("execlp ls");
        exit(1);
    }

    /* Second child */
    child2 = fork();

    if (child2 < 0) {
        perror("fork");
```

```c
    /* Second child */
    child2 = fork();

    if (child2 < 0) {
        perror("fork");
        return 1;
    }

    if (child2 == 0) {

        /* Redirect stdin from pipe */
        dup2(pipefd[0], STDIN_FILENO);

        /* Close unused descriptors */
        close(pipefd[0]);
        close(pipefd[1]);

        /* Execute wc -l */
        execlp("wc", "wc", "-l", NULL);

        perror("execlp wc");
        exit(1);
    }

    /* Parent closes both pipe ends */
    close(pipefd[0]);
    close(pipefd[1]);

    /* Wait for both children */
    waitpid(child1, NULL, 0);
    waitpid(child2, NULL, 0);

    printf("Pipeline execution completed.\n");

    return 0;
}
```

Task 2: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```
kiran@k:~/OSSP$ cd ~/OSSP
kiran@k:~/OSSP$ nano multi_pipe.c
kiran@k:~/OSSP$ gcc multi_pipe.c -o multi_pipe
kiran@k:~/OSSP$ ./multi_pipe
Multiple Pipe Shell
Enter number of commands (2-10): 3
Command 1: ls
Command 2: grep ".c"
Command 3: wc -l
27

All pipeline processes completed.
Resources cleaned up successfully.
kiran@k:~/OSSP$
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20

void parse_command(char *command, char **args) {

    int i = 0;

    args[i] = strtok(command, " \t\n");

    while (args[i] != NULL && i < MAX_ARGS - 1) {

        i++;

        args[i] = strtok(NULL, " \t\n");
    }

    args[i] = NULL;
}

int main() {

    int n;

    printf("Enter number of commands: ");
    scanf("%d", &n);

    getchar();

    if (n < 2 || n > MAX_COMMANDS) {

        printf("Number of commands must be between 2 and %d.\n",
               MAX_COMMANDS);

        return 1;
    }

    char commands[MAX_COMMANDS][256];

    /* Read commands */
    for (int i = 0; i < n; i++) {

        printf("Command %d: ", i + 1);

        fgets(commands[i],
              sizeof(commands[i]),
              stdin);
```

```
GNU nano 8.7.1
        commands[i][strcspn(commands[i], "\n")] = '\0';
    }

    /*
     * Number of pipes is one less than
     * number of commands.
     */
    int pipes[MAX_COMMANDS - 1][2];

    /* Create pipes */
    for (int i = 0; i < n - 1; i++) {

        if (pipe(pipes[i]) == -1) {

            perror("pipe");
            return 1;
        }
    }

    pid_t pids[MAX_COMMANDS];

    /* Create processes */
    for (int i = 0; i < n; i++) {

        pids[i] = fork();

        if (pids[i] < 0) {

            perror("fork");
            return 1;
        }

        if (pids[i] == 0) {

            /*
             * If this is not the first command,
             * connect stdin to previous pipe.
             */
            if (i > 0) {

                dup2(pipes[i - 1][0],
                     STDIN_FILENO);
            }

            /*
             * If this is not the last command,
             * connect stdout to next pipe.
             */
            if (i < n - 1) {

                dup2(pipes[i][1],
                     STDOUT_FILENO);
```

```
                dup2(pipes[i][1],
                     STDOUT_FILENO);
            }

            /* Close all pipe descriptors */
            for (int j = 0; j < n - 1; j++) {

                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            char *args[MAX_ARGS];

            parse_command(commands[i], args);

            if (args[0] == NULL)
                exit(1);

            execvp(args[0], args);

            perror("execvp");
            exit(1);
        }
    }

    /* Parent closes all pipe descriptors */
    for (int i = 0; i < n - 1; i++) {

        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    /* Wait for all processes */
    for (int i = 0; i < n; i++) {

        waitpid(pids[i], NULL, 0);
    }

    printf("All pipeline processes completed.\n");

    return 0;
}
```